UNIVERSITY OF DERBY

College of Science and Engineering

Timed In-Class Test Worked Solutions

OPERATIONAL RESEARCH – 5CM522

DATE: MAY 2026

These solutions match the current live paper.

Source paper:

- `paper/in-class-test-2026-05.tex`

Runnable verification helpers are in:

- `solutions/in_class_test_code.py`
- `solutions/worked_solution_snippets.py`

To verify the whole solution pack quickly:

```
python solutions/worked_solution_snippets.py
```

1. Linear Programming

a) Model formulation

Let:

$$x = \text{number of Standard panels}, \qquad y = \text{number of Premium panels}$$

Model:

$$\max Z = 42x + 36y$$

subject to

$$4x + 2y \leq 84$$

$$2x + 3y \leq 60$$

$$y \leq 18$$

$$x, y \geq 0$$

For marking, either assumption should be accepted if stated clearly:

- continuous non-negative decision variables, giving the LP solution below
- whole-number decision variables, since panels are indivisible in practice

b) Optimal weekly production plan

At the optimum, printing and finishing are both binding:

$$4x + 2y = 84, \qquad 2x + 3y = 60$$

Solving gives:

$$x = 16.5, \qquad y = 9$$

Maximum weekly contribution:

$$Z = 42(16.5) + 36(9) = 1017$$

c) Binding constraints and unused capacity

Binding/slack:

- Printing slack = 0
- Finishing slack = 0
- Premium market slack = $18 - 9 = 9$

d) Value of extra capacity

Extra-capacity interpretation:

- With 6 extra printing hours, contribution rises to 1057.5
- Improvement = 40.5, so value per printing hour = 6.75
- With 6 extra finishing hours, contribution rises to 1062
- Improvement = 45, so value per finishing hour = 7.50

So finishing is more valuable at the current optimum, and the maximum worthwhile price is:

£7.50 per finishing hour, for up to the first 6 hours

e) Premium-share requirement

Premium-share requirement:

$$y \geq 0.3(x + y)$$

which rearranges to

$$0.7y \geq 0.3x \qquad \Longleftrightarrow \qquad y \geq \frac{3x}{7}$$

At the original optimum:

$$\frac{y}{x+y} = \frac{9}{16.5+9} = \frac{9}{25.5} \approx 35.29\%$$

So the original optimum is still feasible, and the optimal plan remains unchanged.

If a student explicitly assumes integer production quantities, the corresponding answers are:

- Optimal whole-number plan: $x = 17$, $y = 8$
- Contribution: $Z = 42(17) + 36(8) = 1002$
- Slack values:
  - Printing slack = 0
  - Finishing slack = $60 - (2(17) + 3(8)) = 2$
  - Premium market slack = $18 - 8 = 10$
- With 6 extra printing hours, the best integer contribution is 1050
- With 6 extra finishing hours, the best integer contribution is 1062
- So finishing is still more valuable over this 6-hour comparison. The average gain is £8 per printing hour versus £10 per finishing hour, but this should not be treated as a constant shadow price in the integer model.
- The 30% Premium requirement is still satisfied because $\frac{8}{17+8} = 32\%$, so the integer optimum also remains unchanged.

Verification snippet:

```
from worked_solution_snippets import q1_results
print(q1_results())
```

2. Binary Integer Programming

a) Model formulation

Let $x_A, \ldots, x_F$ be binary variables where $x_s = 1$ if site s is opened.

Baseline model:

$$\min Z = 16x_A + 11x_B + 15x_C + 14x_D + 13x_E + 10x_F$$

Coverage constraints:

$$x_A + x_B \geq 1$$

$$x_A + x_C + x_E \geq 1$$

$$x_B + x_C + x_F \geq 1$$

$$x_A + x_D + x_F \geq 1$$

$$x_C + x_D + x_E \geq 1$$

$$x_D + x_E + x_F \geq 1$$

$$x_s \in \{0, 1\}$$

b) Baseline optimal solution

Baseline optimum:

$$\text{Open } B, E, F$$

Minimum weekly cost:

$$34$$

c) Redundant coverage in the baseline solution

Redundancy in the baseline solution:

- District 3 is covered by B and F
- District 6 is covered by E and F

Operational value:

- This redundancy gives resilience if one selected site is temporarily unavailable.

d) Revised model and impact of restrictions

Revised model adds:

$$x_D + x_E + x_F \geq 2$$

$$x_B + x_F \leq 1$$

Revised optimum:

$$\text{Open } B, D, E$$

Minimum weekly cost:

$$38$$

e) One realistic extension

One realistic extension:

- Add site capacity limits or maximum response times.

Extra data needed:

- expected demand by district
- travel times from each site to each district
- maximum launch capacity at each site

Verification snippet:

```
from worked_solution_snippets import q2_results
print(q2_results())
```

3. Transhipment

a) Transhipment model formulation

Let:

$$x_{ph} = \text{flow from depot } p \text{ to hub } h$$

$$y_{hc} = \text{flow from hub } h \text{ to clinic } c$$

Objective:

- Minimise total inbound and outbound transport cost.

Constraints:

- Supply constraints at depots
- Demand constraints at clinics
- Flow balance at each hub
- Non-negativity

b) Baseline optimal shipment plan

Baseline optimal solution:

- $P_1 \to H_1 = 100$
- $P_2 \to H_2 = 120$
- $P_3 \to H_2 = 40$
- $H_1 \to C_1 = 70$
- $H_1 \to C_4 = 30$
- $H_2 \to C_2 = 60$
- $H_2 \to C_3 = 80$
- $H_2 \to C_4 = 20$

Baseline total cost:

1950

Unused supply:

40 units at P_3

c) Disrupted scenario

Disrupted model adds:

$$\sum_{c} y_{H_1 c} \leq 130$$

$$y_{H_1 C_2} \geq 30$$

Revised optimal solution:

- $P_1 \rightarrow H_1 = 100$
- $P_2 \rightarrow H_2 = 120$
- $P_3 \rightarrow H_2 = 40$
- $H_1 \rightarrow C_1 = 20$
- $H_1 \rightarrow C_2 = 30$
- $H_1 \rightarrow C_4 = 50$
- $H_2 \rightarrow C_1 = 50$
- $H_2 \rightarrow C_2 = 30$
- $H_2 \rightarrow C_3 = 80$

Revised total cost:

$$2040$$

Interpretation:

- Forcing at least 30 units for clinic C_2 through H_1 changes the network pattern and increases cost.

d) Emergency direct route from P_3 to C_4

Emergency direct route extension:

- Add direct decision variable d for $P_3 \rightarrow C_4$
- Bounds: $0 \leq d \leq 20$
- Add $9d$ to the objective
- The 50 comes from clinic C_4's original demand, so replace the C_4 demand constraint with $y_{H_1 C_4} + y_{H_2 C_4} + d = 50$
- Reduce available P_3 hub shipments accordingly, so $x_{P_3 H_1} + x_{P_3 H_2} + d \leq 80$

Optimal direct-shipping decision:

$$d = 20$$

Revised non-zero shipments:

- $P_1 \rightarrow H_1 = 100$
- $P_2 \rightarrow H_2 = 120$
- $P_3 \rightarrow H_2 = 20$
- $P_3 \rightarrow C_4 = 20$
- $H_1 \rightarrow C_1 = 70$

- $H_1 \to C_2 = 30$
- $H_2 \to C_2 = 30$
- $H_2 \to C_3 = 80$
- $H_2 \to C_4 = 30$

Revised total cost with emergency direct route:

$$2020$$

So the direct route is worthwhile because it reduces cost by 20. It is cheaper than sending those units via H_2, since $P_3 \to H_2 \to C_4$ costs $4 + 6 = 10$ per unit while the emergency route costs 9 per unit.

Verification snippet:

```
from worked_solution_snippets import q3_results
print(q3_results())
```

4. Simulation

a) Main DES elements

Main DES elements:

- Entities: students
- Resource: technicians
- Events: arrivals, service start, service completion
- Stochastic inputs: interarrival times and service times
- Measures: number served, average wait, maximum wait, percentage waiting less than 10 minutes

b) Baseline SimPy run with 2 technicians

Baseline one-run results with 2 technicians:

- Students served = 71
- Average wait = 5.2208 minutes
- Maximum wait = 46.4285 minutes
- Percentage under 10 minutes = 87.3239%

c) Busy-period staffing comparison

Busy-period results at 13 students per hour:

Technicians	Served	Avg wait	Max wait	% under 10 min
2	110	43.8859	170.5482	30.9091
3	105	4.0800	28.1272	83.8095
4	105	0.1727	3.2712	100.0000

Recommendation from part (c):

- 3 technicians is the minimum staffing level that meets the primary target of average wait below 8 minutes.
- It does not meet the secondary target of at least 85% under 10 minutes.

d) Mixed service-time extension

Mixed-service extension:

- Standard queries: 75%, mean service 6 minutes
- Complex queries: 25%, mean service 14 minutes
- Keep arrivals at 13 per hour
- For each arrival, sample whether the case is standard or complex
- Then sample service time from the corresponding exponential distribution

One run with 3 technicians gives:

- Students served = 84
- Complex cases = 29
- Average wait = 2.1404 minutes
- Maximum wait = 14.5512 minutes
- Percentage under 10 minutes = 91.6667%

So the 3-technician recommendation still looks reasonable on this run.

e) Why one simulation run is not enough

Why one run is not enough:

- Output depends on random variation.
- One run can be unusually good or bad.
- Repeated runs allow a mean estimate and confidence interval.

Verification snippet:

```
from worked_solution_snippets import q4_results
print(q4_results())
```

5. Forecasting

a) Centred moving averages and trend

Using periods 1–12, the centred 4-quarter moving averages can be calculated only for periods 3–10:

Period	Actual	MA4	CMA
2	505	482.5000	
3	455	490.0000	486.2500
4	590	501.2500	495.6250
5	410	512.5000	506.8750
6	550	526.2500	519.3750
7	500	535.0000	530.6250
8	645	547.5000	541.2500
9	445	557.5000	552.5000
10	600	572.5000	565.0000

Fitting a straight line to the CMA values for periods 3–10 gives the estimated trend:

$$\text{Trend} = 451.1756 + 11.3095t$$

b) Seasonal indices

Estimated seasonal indices:

Additive:

$$Q1 = -101.8750, \quad Q2 = 33.1250, \quad Q3 = -30.6250, \quad Q4 = 99.3750$$

Multiplicative:

$$Q1 = 0.8076, \quad Q2 = 1.0611, \quad Q3 = 0.9396, \quad Q4 = 1.1917$$

c) Validation-set comparison using MSE

Validation-set accuracy on periods 13–16 uses:

$$\text{MSE} = \frac{1}{n}\sum_{t=1}^{n} e_t^2$$

where

$$e_t = \text{Actual}_t - \text{Forecast}_t$$

Results:

$$\text{MSE}_{\text{additive}} = 558.4018$$

$$\text{MSE}_{\text{multiplicative}} = 184.4082$$

So the better model is multiplicative.

d) Forecasts for 2023 and recommendation

Forecasts for 2023 using the multiplicative model:

Period	Quarter	Forecast
17	Q1	519.6556
18	Q2	694.7350
19	Q3	625.7974
20	Q4	807.2476

Recommendation:

- Use the multiplicative model.
- It gives the lower validation MSE.
- The seasonal swing appears to rise with the level of the series, which supports a multiplicative interpretation.

Observation to check:

- Period 16 with demand 775 is the largest observation and worth checking before operational use.

Verification snippet:

```
from worked_solution_snippets import q5_results
print(q5_results())
```

6. Scheduling

Part A: Johnson's Rule

a) Johnson's Rule sequence

Processing times:

$$J_1 = (8, 4), \quad J_2 = (3, 9), \quad J_3 = (6, 5), \quad J_4 = (7, 2), \quad J_5 = (4, 6)$$

Johnson steps:

(a) Smallest time is 2 on J_4 on Machine 2, so place J_4 last.

(b) Next smallest is 3 on J_2 on Machine 1, so place J_2 first.

(c) Next smallest is 4 on J_5 on Machine 1, so place J_5 next.

(d) Next smallest remaining is 4 on J_1 on Machine 2, so place J_1 in the latest remaining slot.

(e) J_3 fills the remaining middle slot.

Optimal Johnson sequence:

$$J_2, J_5, J_3, J_1, J_4$$

b) Gantt chart and minimum makespan

Gantt-style timing:

Job	M1 start	M1 finish	M2 start	M2 finish
J_2	0	3	3	12
J_5	3	7	12	18
J_3	7	13	18	23
J_1	13	21	23	27
J_4	21	28	28	30

Minimum makespan:

30

c) Comparison with the original order

Original-order schedule J_1, J_2, J_3, J_4, J_5 gives makespan:

34

Improvement:

4 time units

Part B: Job Shop Solution

d) Job shop model, schedule and makespan

Use an optimisation model such as OR-Tools CP-SAT with:

- Objective: minimise makespan
- Decision variables: start times, end times, or interval tasks for each operation
- Precedence constraints: each task must follow the previous task in the same job
- Machine-capacity constraints: no overlap on each machine

One optimal baseline schedule is:

Job	Task	Machine	Start	End
A	1	M_1	0	3
B	1	M_2	0	4
C	1	M_3	0	3
A	2	M_3	3	8
B	2	M_1	4	6
C	2	M_2	4	8
D	1	M_1	6	8
B	3	M_3	8	14
C	3	M_1	8	11
D	2	M_2	8	14
A	3	M_2	14	16
D	3	M_3	14	18

So the minimum makespan is:

18

Short interpretation:

- Precedence is enforced because each job's next operation cannot start before the previous one finishes.
- Non-overlap is enforced because each machine can process at most one operation at a time.

e) Revised feasibility with deadline and machine downtime

Part (e) revised model:

- Add due-date constraint: completion time of job D must be ≤ 15
- Add machine downtime on M_2 from time 6 to 9, for example by inserting a blocking interval

Re-solving the model gives:

infeasible

Operational interpretation:

- The new restrictions remove too much flexibility from the schedule.
- Job D needs 6 uninterrupted time units on M_2 after first visiting M_1.
- Because M_2 is unavailable from time 6 to 9, job D cannot both complete its M_2 operation and still finish the full job by time 15.
- A solver reporting infeasibility means there is no schedule that satisfies all constraints simultaneously.

Verification snippet:

```
from worked_solution_snippets import q6_results
print(q6_results())
```